

Rate Monotonic Scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
typedef struct {
    int id;
    int burst;
    int period;
    int remaining_time;
    int is_ready;
} Task;
```

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

```
int find_lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

```
int main() {
    int n;

    printf("1BM24CS361, Nitya R Swadi\n");

    printf("Enter the number of processes: ");
```

```

if (scanf("%d", &n) != 1 || n <= 0) {
    printf("Invalid number of processes.\n");
    return 1;
}

```

```

Task tasks[n];

```

```

printf("Enter the CPU burst times:\n");
for (int i = 0; i < n; i++) {
    tasks[i].id = i + 1;
    scanf("%d", &tasks[i].burst);
}

```

```

printf("Enter the time periods:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &tasks[i].period);
    tasks[i].remaining_time = 0;
    tasks[i].is_ready = 0;
}

```

```

for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (tasks[j].period > tasks[j + 1].period) {
            Task temp = tasks[j];
            tasks[j] = tasks[j + 1];
            tasks[j + 1] = temp;
        }
    }
}

```

```

int hyperperiod = tasks[0].period;
for (int i = 1; i < n; i++) {
    hyperperiod = find_lcm(hyperperiod, tasks[i].period);
}
printf("LCM=%d\n\n", hyperperiod);

```

```

// Rate Monotone Table Output
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", tasks[i].id, tasks[i].burst, tasks[i].period);
}
printf("\n");

double utilization = 0.0;
for (int i = 0; i < n; i++) {
    utilization += (double)tasks[i].burst / tasks[i].period;
}
double bound = n * (pow(2.0, 1.0 / n) - 1.0);

printf("%f <= %f =>", utilization, bound);
if (utilization <= bound) {
    printf("true\n");
} else {
    printf("false (System might not be schedulable, but simulating anyway)\n");
}
printf("Scheduling occurs for %d ms\n\n", hyperperiod);

int current_time = 0;
int last_running_task_id = -2;

while (current_time < hyperperiod) {

    for (int i = 0; i < n; i++) {
        if (current_time % tasks[i].period == 0) {
            tasks[i].remaining_time = tasks[i].burst;
            tasks[i].is_ready = 1;
        }
    }

    int selected_task_index = -1;
    for (int i = 0; i < n; i++) {

```

```

        if (tasks[i].is_ready && tasks[i].remaining_time > 0) {
            selected_task_index = i;
            break;
        }
    }

    if (selected_task_index != -1) {
        if (last_running_task_id != tasks[selected_task_index].id) {
            printf("%dms onwards: Process %d running\n", current_time,
tasks[selected_task_index].id);
            last_running_task_id = tasks[selected_task_index].id;
        }

        tasks[selected_task_index].remaining_time--;

        if (tasks[selected_task_index].remaining_time == 0) {
            tasks[selected_task_index].is_ready = 0;
        }
    } else {

        if (last_running_task_id != -1) {
            printf("%dms onwards: CPU is idle\n", current_time);
            last_running_task_id = -1;
        }
    }

    current_time++;
}

return 0;
}

```

OUTPUT:

C:\Users\Nitya\Downloads\rm X + v

IBM24CS361, Nitya R Swadi
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:

PID	Burst	Period
1	20	50
2	35	100

$0.750000 \leq 0.828427 \Rightarrow \text{true}$
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0) execution time : 21.226 s
Press any key to continue.